

Your grade: 90%

Your latest: 90% • Your highest: 90% • To pass you need at least 80%. We keep your highest score.

Next item →

1. What does a neuron compute?

1 / 1 point

- ☒ A neuron computes a linear function $z = Wx + b$ followed by an activation function
- ☐ A neuron computes a function g that scales the input x linearly ($Wx + b$)
- ☐ A neuron computes the mean of all features before applying the output to an activation function
- ☐ A neuron computes an activation function followed by a linear function $z = Wx + b$

✔ Nice work

 Correct, we generally say that the output of a neuron is $a = g(Wx + b)$ where g is the activation function (sigmoid, tanh, ReLU, ...).

2. Which of these is the "Logistic Loss"?

1 / 1 point

- ☒ $\mathcal{L}^{(i)}(\hat{y}^{(i)}, y^{(i)}) = -(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$
- ☐ $\mathcal{L}^{(i)}(\hat{y}^{(i)}, y^{(i)}) = |y^{(i)} - \hat{y}^{(i)}|$
- ☐ $\mathcal{L}^{(i)}(\hat{y}^{(i)}, y^{(i)}) = |y^{(i)} - \hat{y}^{(i)}|^2$
- ☐ $\mathcal{L}^{(i)}(\hat{y}^{(i)}, y^{(i)}) = \max(0, y^{(i)} - \hat{y}^{(i)})$

✔ Nice work

Correct, this is the logistic loss you've seen in lecture!

 3. Suppose `img` is a (32,32,3) array, representing a 32x32 image with 3 color channels red, green and blue. How do you reshape this into a column vector x ?

1 / 1 point

- ☐ `x = img.reshape((1,32*32,3))`
- ☐ `x = img.reshape((3,32*32))`
- ☐ `x = img.reshape((32*32,3))`

3. Suppose `img` is a `(32,32,3)` array, representing a 32x32 image with 3 color channels red, green and blue. How do you reshape this into a column vector x ?

1 / 1 point

- ☐ `x = img.reshape((1,32*32,3))`
- ☐ `x = img.reshape((3,32*32))`
- ☐ `x = img.reshape((32*32,3))`
- ☒ `x = img.reshape((32*32*3,1))`

 Nice work

4. Consider the following random arrays a and b , and c :

1 / 1 point

$$a = np.random.randn(2, 3) \# a.shape = (2, 3)$$
$$b = np.random.randn(2, 1) \# b.shape = (2, 1)$$
$$c = a + b$$

What will be the shape of c ?

- ☐ `c.shape = (3, 2)`
- ☐ `c.shape = (2, 1)`
- ☒ `c.shape = (2, 3)`
- ☐ The computation cannot happen because the sizes don't match. It's going to be "Error"!

 Nice work

Yes! This is broadcasting. b (column vector) is copied 3 times so that it can be summed to each column of a .

5. Consider the two following random arrays a and b :

1 / 1 point

$$a = np.random.randn(4, 3) \# a.shape = (4, 3)$$
$$b = np.random.randn(3, 2) \# b.shape = (3, 2)$$

5. Consider the two following random arrays a and b :

$$a = np.random.randn(4, 3) \# a.shape = (4, 3)$$
$$b = np.random.randn(3, 2) \# b.shape = (3, 2)$$
$$c = a * b$$

What will be the shape of c ?

- ☐ c.shape = (4,2)
- ☐ c.shape = (3, 3)
- ☐ c.shape = (4, 3)
- ☒ The computation cannot happen because the sizes don't match. It's going to be "Error"!

✔ Nice work

Indeed! In numpy the "*" operator indicates element-wise multiplication. It is different from "np.dot()". If you would try "c = np.dot(a,b)" you would get c.shape = (4, 2).

1 / 1 point

6. Suppose our input batch consists of 8 grayscale images, each of dimension 8x8. We reshape these images into feature column vectors $\mathbf{x}^j$. Remember that $X = [\mathbf{x}^{(1)} \mathbf{x}^{(2)} \dots \mathbf{x}^{(8)}]$. What is the dimension of X ?

- ☐ (8, 8, 8)
- ☐ (8, 64)
- ☐ (512, 1)
- ☒ (64, 8)

✔ Nice work

Yes. After converting the 8x8 gray scale images to a column vector we get a vector of size 64, thus X has dimension (64, 8).

1 / 1 point

7. Recall that $np.dot(a, b)$ performs a matrix multiplication on a and b , whereas $a * b$ performs an element-wise multiplication.

1 / 1 point

7. Recall that $np.dot(a, b)$ performs a matrix multiplication on a and b , whereas $a * b$ performs an element-wise multiplication.

1 / 1 point

Consider the two following random arrays a and b :

$$a = np.random.randn(12288, 150) \# a.shape = (12288, 150)$$
$$b = np.random.randn(150, 45) \# b.shape = (150, 45)$$
$$c = np.dot(a, b)$$

What is the shape of c ?

- ☐ $c.shape = (150, 150)$
- ☒ $c.shape = (12288, 45)$
- ☐ The computation cannot happen because the sizes don't match. It's going to be "Error"!
- ☐ $c.shape = (12288, 150)$

✔ Nice work

Correct, remember that a $np.dot(a, b)$ has shape (number of rows of a , number of columns of b). The sizes match because :

"number of columns of $a = 150 =$ number of rows of b "

8. Consider the following code snippet:

0 / 1 point

$$a.shape = (4, 3)$$
$$b.shape = (4, 1)$$

```
for i in range(3):
```

```
    for j in range(4):
```

```
        c[i][j] = a[j][i] + b[j]
```

"number of columns of a = 150 = number of rows of b"

8. Consider the following code snippet:

0 / 1 point

```
a.shape = (4, 3)
```

```
b.shape = (4, 1)
```

```
for i in range(3):
```

```
    for j in range(4):
```

```
        c[i][j] = a[j][i] + b[j]
```

How do you vectorize this?

☐ `c = a + b.T`

☐ `c = a.T + b`

☒ `c = a + b`

☐ `c = a.T + b.T`

⊗ Try again

No. The `a[j][i]` being assigned to `a[i][j]` indicates that we are using `a.T`.

9. Consider the following arrays:

1 / 1 point

```
a = np.array([[1, 1], [1, -1]])
```

```
b = np.array([[2], [3]])
```

```
c = a + b
```

Which of the following arrays is stored in `c`?

9. Consider the following arrays:

1 / 1 point

$a = np.array([[1, 1], [1, -1]])$

$b = np.array([[2], [3]])$

$c = a + b$

Which of the following arrays is stored in c ?

☐ $\begin{pmatrix} 3 & 4 \\ 3 & 2 \end{pmatrix}$

☒ $\begin{pmatrix} 3 & 3 \\ 4 & 2 \end{pmatrix}$

☐ The computation cannot happen because the sizes don't match. It's going to be an "Error"!

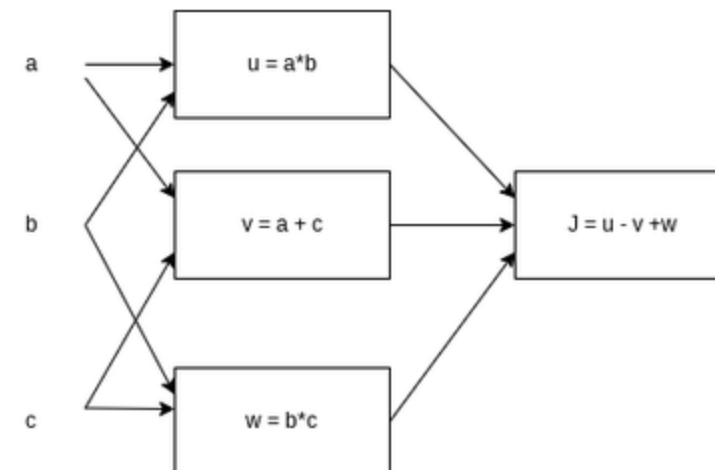
☐ $\begin{pmatrix} 3 & 3 \\ 3 & 1 \\ 4 & 4 \\ 5 & 2 \end{pmatrix}$

 **Nice work**

Yes. The array b is a column vector. This is copied two times and added to the array a to construct the array c .

10. Consider the following computational graph.

1 / 1 point



What is the output of J ?

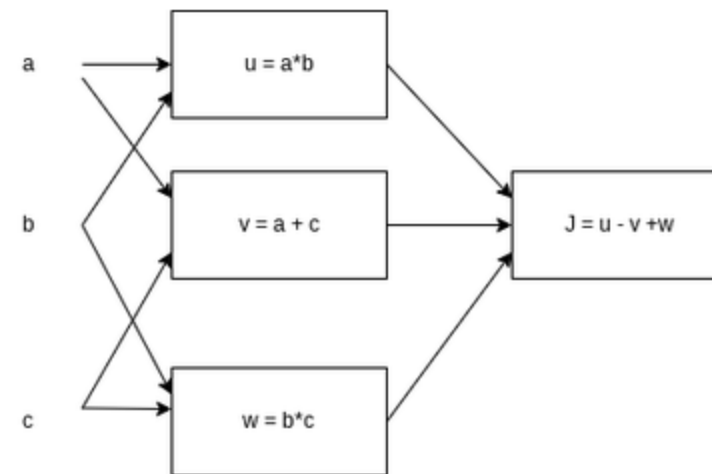
$$\begin{pmatrix} 3 & 1 \\ 4 & 4 \\ 5 & 2 \end{pmatrix}$$

✔ Nice work

Yes. The array b is a column vector. This is copied two times and added to the array a to construct the array c .

10. Consider the following computational graph.

1 / 1 point



What is the output of J ?

- ☐ $ab + bc + ac$
- ☐ $(a - 1)(b + c)$
- ☐ $(c - 1)(a + c)$
- ☒ $(a + c)(b - 1)$

✔ Nice work

Yes.

$$J = u - v + w = ab - (a + c) + bc = ab - a + bc - c = a(b - 1) + c(b - 1) = (a + c)(b - 1)$$